

UEBUNG_1

Übung 1 : GenerateFlowFile → ReplaceText → UpdateAttribute → PutFile

Apache NiFi 2.8.0 · UI: <https://localhost:8443/nifi> · Login `admin` `N=/Users/grigo/gits/seminars/it-schulungen.com/Apache_NiFi/nifi-2.8.0` Fertiges Referenz-Lab auf der Canvas: “**Uebung 1 - GFF-Replace-Update-Put**”

Lernziel: Inhalt UND Metadaten (Attribute) einer FlowFile verändern und am Ende über **Data Provenance** den kompletten Weg eines einzelnen FlowFiles nachvollziehen.

Zielordner: `$N/data/output/uebung1` > Bewusst **nicht** `/tmp/nifi-uebung1/`, sondern im Projektbaum unter `data/output/uebung1` : > liegt bei den anderen Lab-Ausgaben, übersteht Neustart, und ist leichter aufzuräumen.

Flow:

```
GenerateFlowFile —> ReplaceText —> UpdateAttribute —> PutFile
  (Testdaten)         (Platzhalter)   (Attribute)         (Zielordner)
```

Schritt 1 : GenerateFlowFile erzeugt Testdaten (alle 30 sec)

Doppelklick → **Properties:** - **Custom Text** = `Auftrag fuer __NAME__ erstellt am ${now():format('yyyy-MM-dd HH:mm:ss')}` - **Data Format** = `Text` - **Batch Size** = `1`

Tab **Scheduling:** **Run Schedule** = `30 sec`

`__NAME__` ist unser **Platzhalter**, den ReplaceText gleich ersetzt. `${now()}` ist Expression Language und wird zur Laufzeit zum aktuellen Zeitstempel.

Schritt 2 : ReplaceText ersetzt den Platzhalter im Inhalt

Doppelklick → **Properties:** - **Replacement Strategy** = `Literal Replace` - **Search Value** = `__NAME__` - **Replacement Value** = `Kunde Mueller` - **Evaluation Mode** = `Entire text`

Wirkt auf den **Inhalt** (Payload) der FlowFile. Aus `Auftrag fuer __NAME__ ...` wird `Auftrag fuer Kunde Mueller ...`.

Schritt 3 : UpdateAttribute ergänzt Metadaten

Doppelklick → **Properties** → mit dem **+** zwei eigene Properties anlegen: - `source = uebung1`
- `priority = normal`

Wichtiger Unterschied: UpdateAttribute ändert **nicht den Inhalt**, sondern die **Attribute** (Metadaten) der FlowFile. Attribute sind Key-Value-Paare, die mit der FlowFile mitreisen (z.B. `filename`, `uuid`, und jetzt zusätzlich `source`, `priority`). Man sieht sie später in der Provenance, nicht im Datei-Inhalt.

Schritt 4 : PutFile schreibt ins Zielverzeichnis

Doppelklick → **Properties:** - **Directory** = `/Users/grigo/gits/seminars/it-schulungen.com/apache_nifi/nifi-2.8.0/data/output/uebung1` - **Create Missing Directories** = `true` - **Conflict Resolution Strategy** = `replace`

Tipp: Mit `Conflict Resolution = replace` überschreibt gleicher Dateiname; mit `ignore/fail` reagiert es anders. Der Dateiname kommt aus dem Attribut `filename` (Standard: die UUID).

Schritt 5 : Verbinden & Relationships

Verbindungen (jeweils Relationship `success`):

```
GenerateFlowFile —success—> ReplaceText —success—> UpdateAttribute —success—> PutFile
```

Terminieren (Tab **Relationships** → `terminate`): - ReplaceText: `failure` - PutFile: `success`, `failure` - (UpdateAttribute hat nur `success` : ist verbunden, nichts zu terminieren.)

Schritt 6 : Validieren, starten, beobachten

1. Alle vier Prozessoren ohne $\triangle$ → starten (Rechtsklick auf die Process Group → **Start**).
2. Nach ~30s erste Datei prüfen:

```
ls -l $N/data/output/uebung1/
cat $N/data/output/uebung1/*
# -> Auftrag fuer Kunde Mueller erstellt am 2026-...
```

3. **Danach stoppen** (GenerateFlowFile produziert sonst alle 30s weiter).
-

Schritt 7 : Provenance öffnen und den Weg eines FlowFiles nachvollziehen

Das ist der Kern dieser Übung.

1. Rechtsklick auf **PutFile** (oder einen beliebigen Prozessor) → **View data provenance**. (Global geht auch: Hamburger-Menü $\equiv$ → **Data Provenance**.)
2. Die Liste zeigt Events. Für unseren Flow typisch:
 - **CREATE** : GenerateFlowFile erzeugt die FlowFile

- **CONTENT_MODIFIED** : ReplaceText ändert den Inhalt
 - **ATTRIBUTES_MODIFIED** : UpdateAttribute setzt `source/priority`
 - **SEND** : PutFile schreibt die Datei
 - **DROP** : FlowFile am Ende verworfen (normal)
3. Auf das ⓘ / Lupen-Symbol einer Zeile klicken → Detail-Dialog:
- Tab **Attributes**: hier `source = uebung1` und `priority = normal` suchen : die in Schritt 3 gesetzten Attribute. Spalte zeigt teils “value” und “previous value”.
 - Tab **Content**: “View” zeigt den Inhalt **vorher/nachher** (z.B. beim CONTENT_MODIFIED-Event `__NAME__` vs. Kunde Mueller).
4. **Lineage-Graph**: im Detail-Dialog (oder Rechtsklick auf ein Event) → **Show Lineage**. Es öffnet sich ein Graph mit allen Stationen der FlowFile. Über die **Zeitachse unten** kann man den Weg **abspielen** : CREATE → CONTENT_MODIFIED → ATTRIBUTES_MODIFIED → SEND → DROP. Rechtsklick auf einen Knoten → “View Details” / “Expand” für mehr.

Aha-Effekt: Provenance ist kein Text-Log, sondern die **vollständige Lebensgeschichte jeder einzelnen FlowFile** : Inhalt, Attribute und Weg, rückverfolgbar bis zur Quelle.

Verifiziertes Ergebnis (im Live-Test bestätigt)

- Datei-Inhalt: Auftrag fuer Kunde Mueller erstellt am 2026-06-02 22:48:20
- Provenance-Events vorhanden: CREATE, ATTRIBUTES_MODIFIED, SEND, DROP, RECEIVE
- Attribute `source=uebung1, priority=normal` an der FlowFile gesetzt.

Reset

```
find $N/data/output/uebung1 -type f -delete
# Process Group stoppen
```

Häufige Stolpersteine

- **Platzhalter bleibt** `__NAME__` → ReplaceText Search Value prüfen, Evaluation Mode = Entire text.
- **Attribute nicht in der Datei** → korrekt! UpdateAttribute schreibt Attribute, nicht in den Inhalt; sie erscheinen in der **Provenance**, nicht im `cat`.
- **Output flutet** → GenerateFlowFile nach dem Test stoppen (läuft alle 30s).
- **Prozessor** △ **invalid** → Relationship nicht verbunden/terminiert (Tab Relationships).